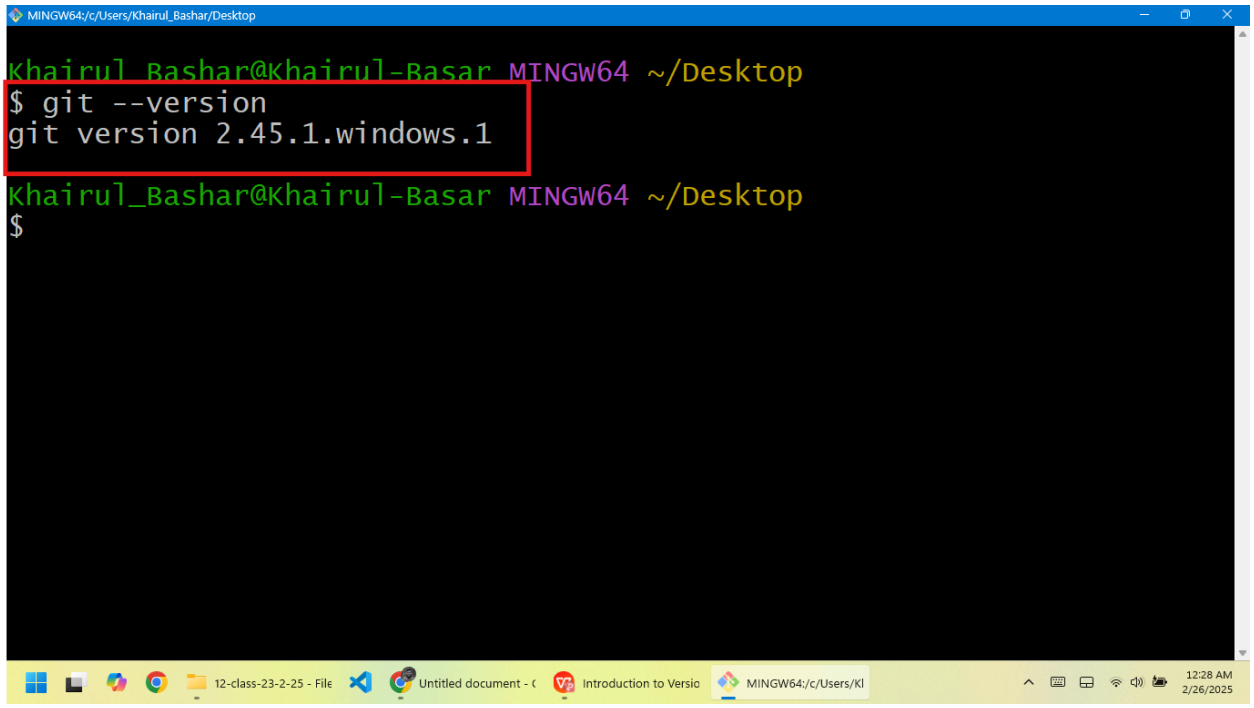
Assignment-12

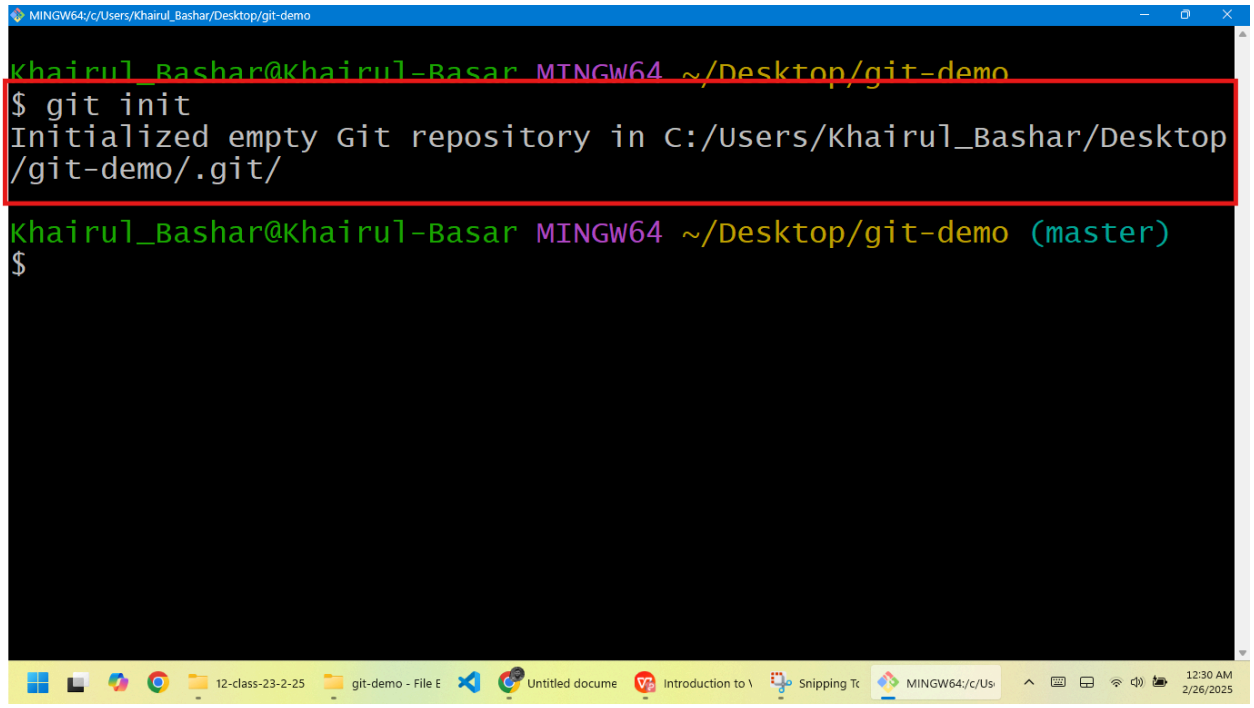
Submitted by Khairul Basar

Step 1: Git Installation

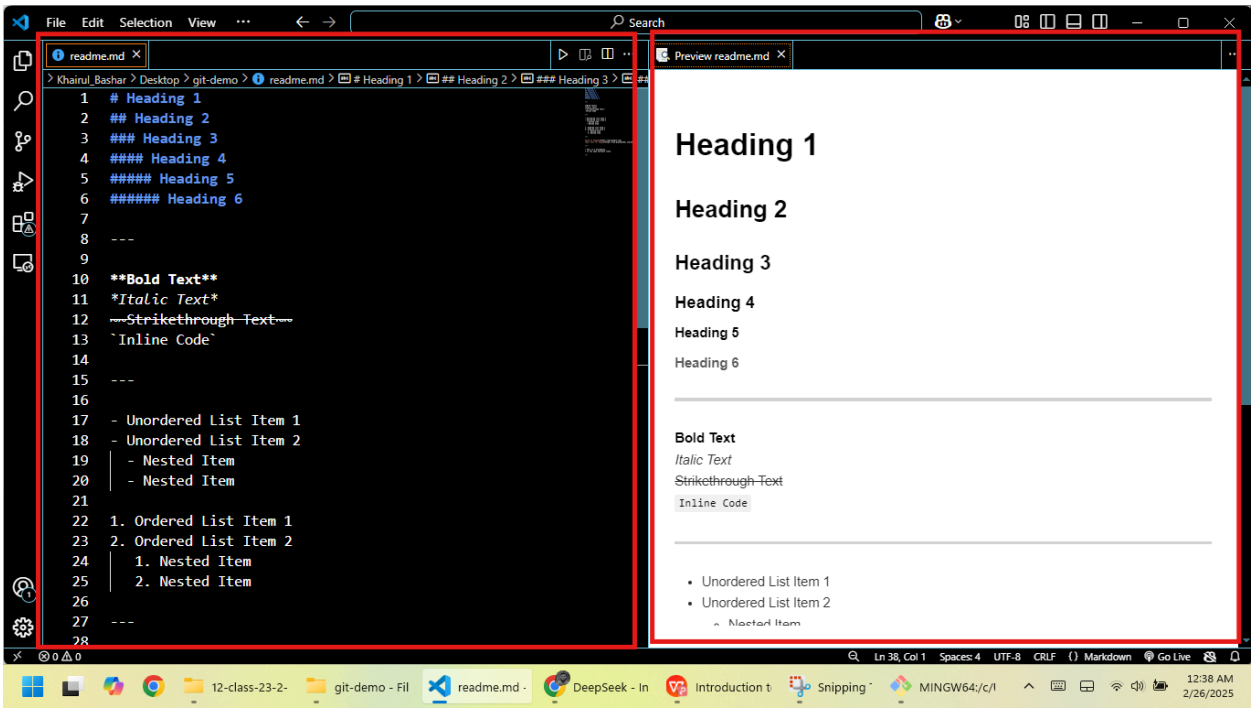


Step 2:

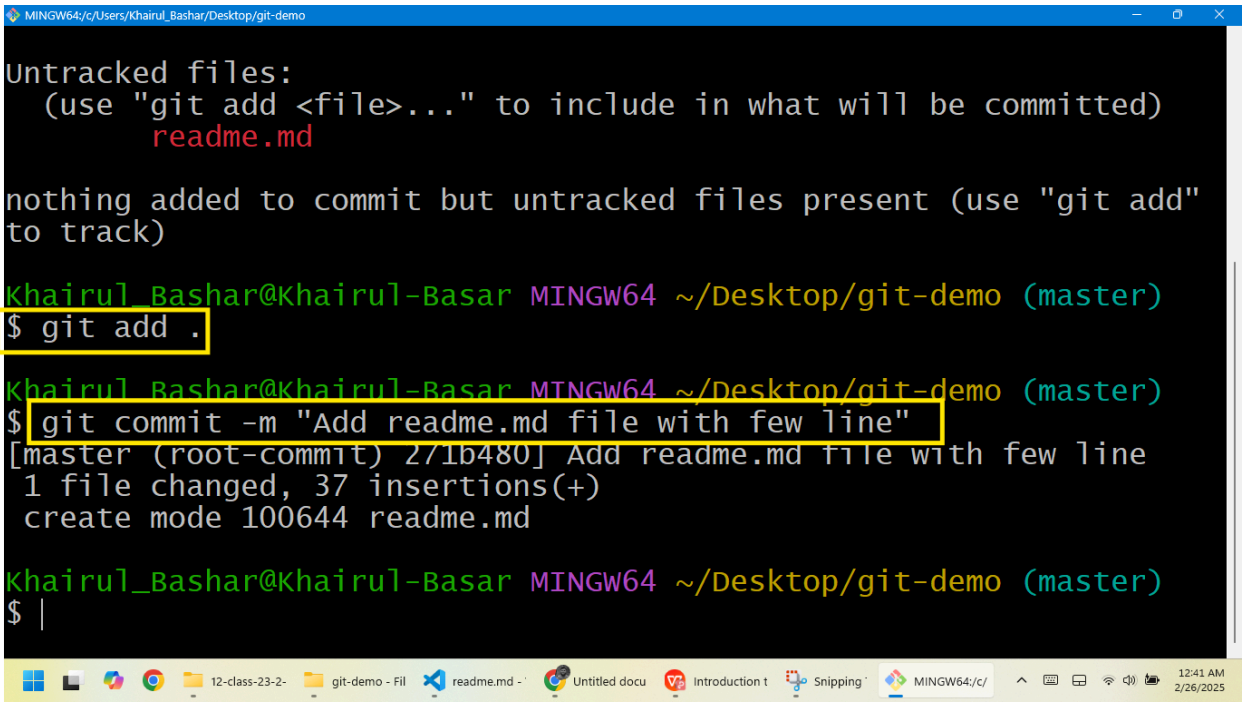
a) Initialize Git:



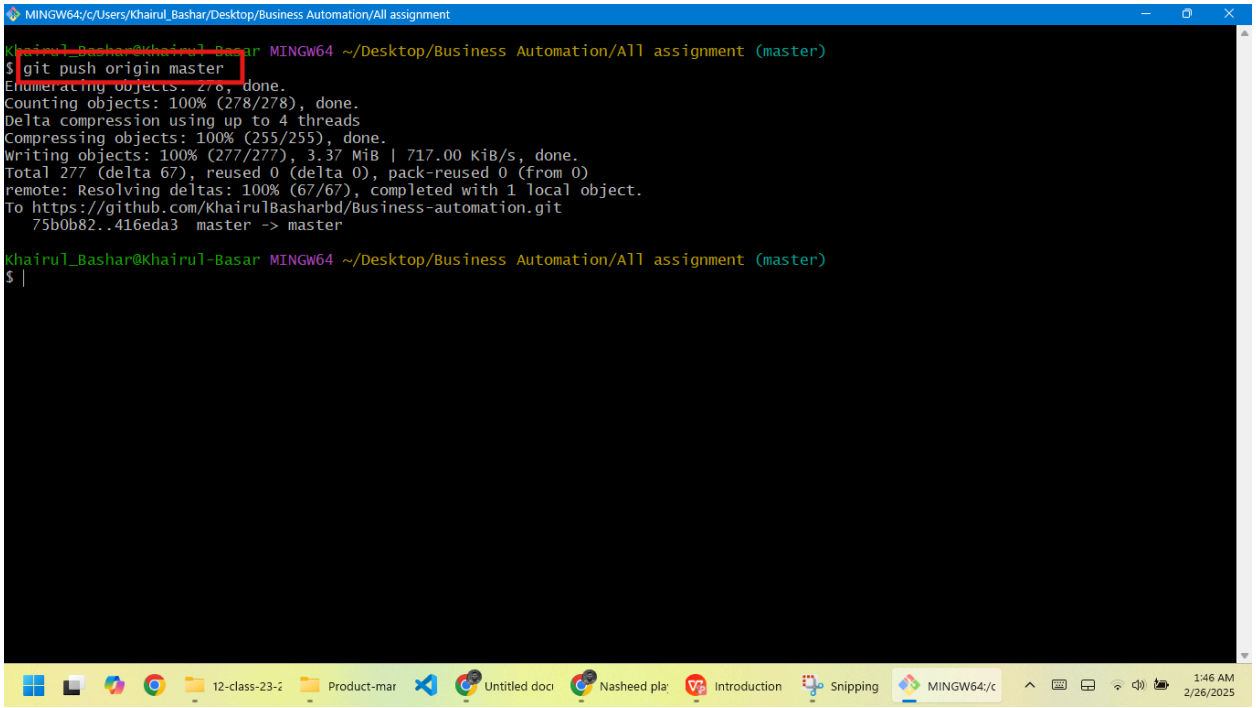
b) Create *readme.md* and add short description



c) Add and commit the change



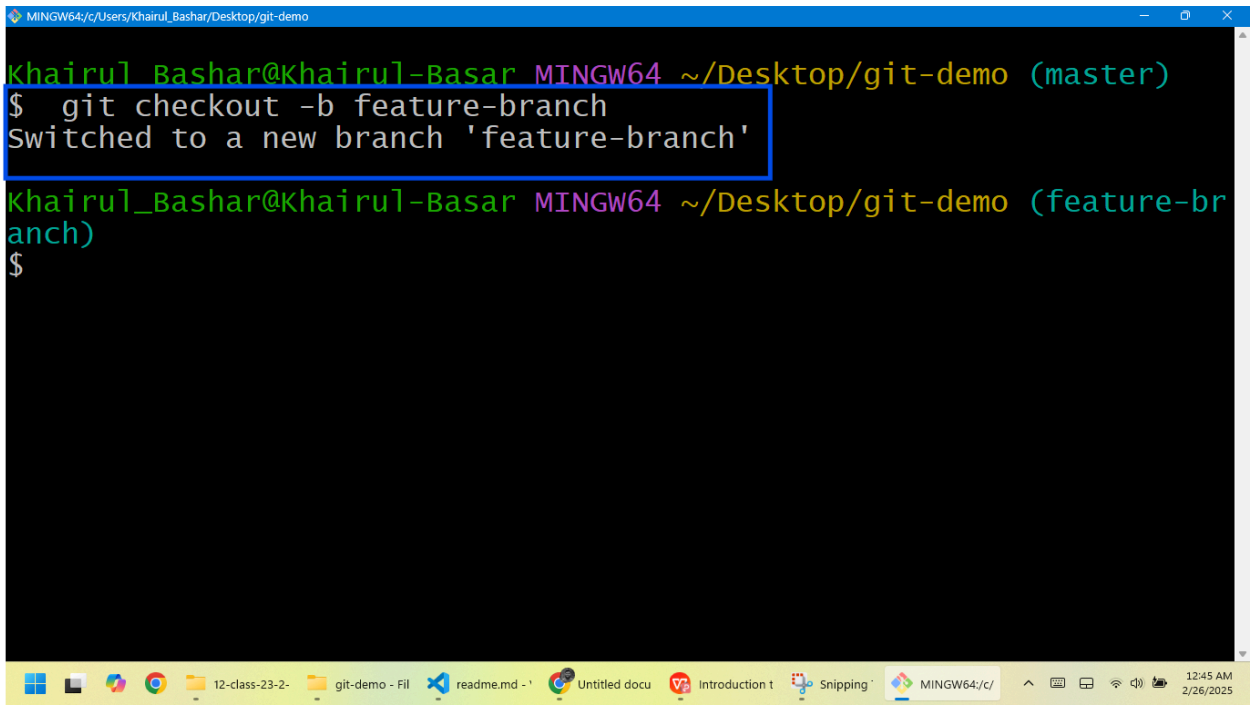
d) Push to github repositories



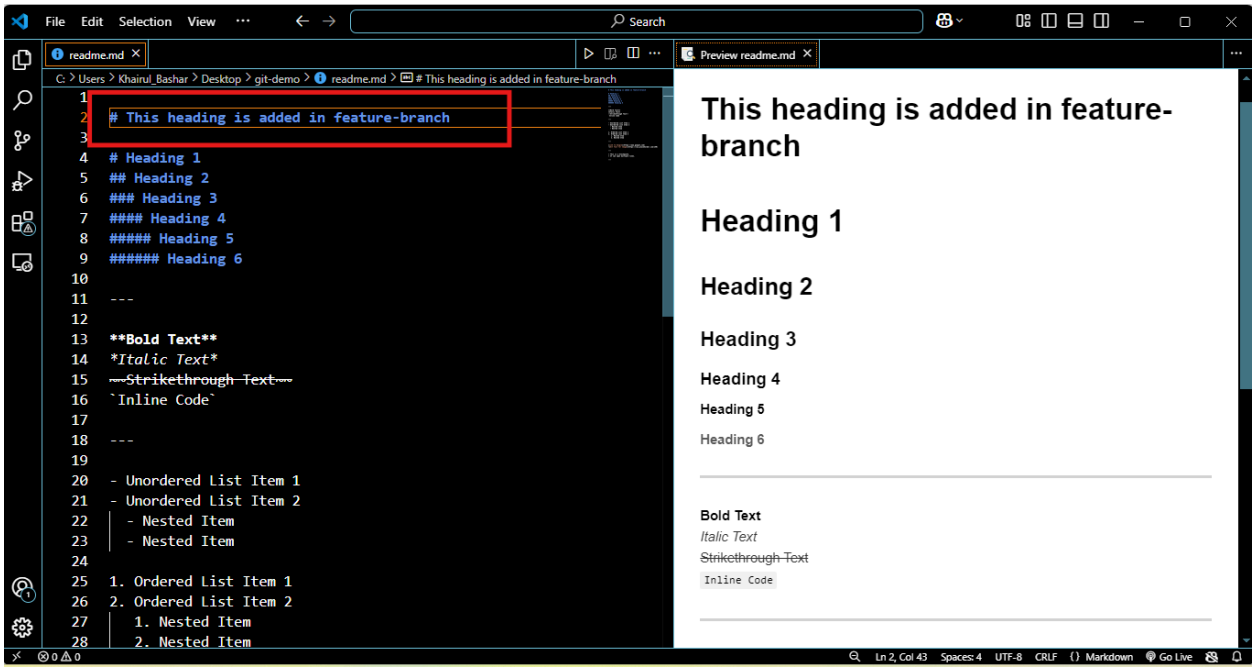
Step 3: Branching and Merging

---

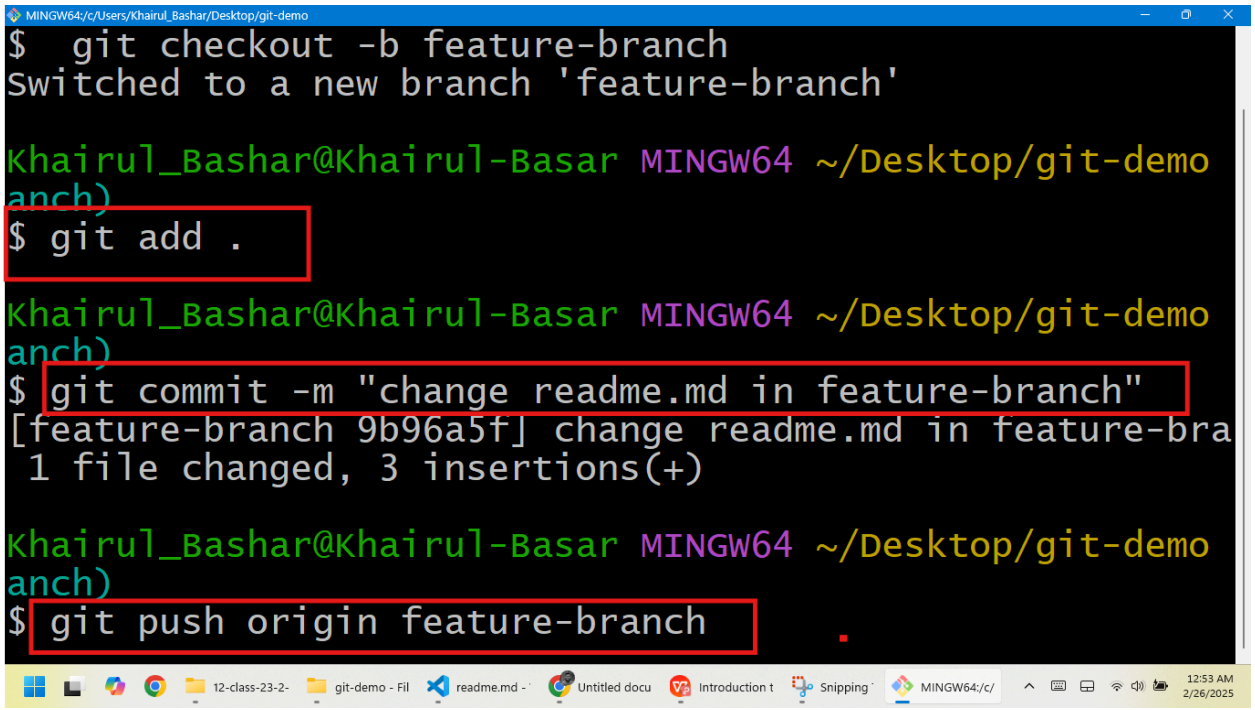
a) Create feature-branch and Switch to it



b) Modify *readme.md* in feature-branch



c) Add, Commit, push to Github and merge



Step 4: Learn About Deployment

---

1. Steps to Deploy a PHP Web Application on a Linux Server Using Jenkins

Deploying a PHP web application on a Linux server using Jenkins involves setting up a CI/CD pipeline to automate the deployment process. Below are the steps:

Step 1: Set Up a Linux Server

- Choose a Linux distribution (e.g., Ubuntu, CentOS).
- Use a cloud provider (e.g., AWS, DigitalOcean, Linode) or a local server.
- SSH into the server:

```
ssh username@server_ip
```

Step 2: Install Required Software

- Update the package manager:

```
sudo apt update && sudo apt upgrade
```

- Install a web server (e.g., Apache or Nginx):

```
sudo apt install apache2
```

- Install PHP and necessary extensions:

```
sudo apt install php libapache2-mod-php php-mysql
```

- Install a database (e.g., MySQL or MariaDB):

```
sudo apt install mysql-server
```

Step 3: Set Up Jenkins

- Install Java (required for Jenkins):

```
sudo apt install openjdk-11-jdk
```

- Add Jenkins repository and install Jenkins:

```
wget -q -O - <https://pkg.jenkins.io/debian/jenkins.io.key | sudo apt-key add -
sudo sh -c 'echo deb <http://pkg.jenkins.io/debian-stable> binary/ >
/etc/apt/sources.list.d/jenkins.list'
sudo apt update
sudo apt install jenkins
```

- Start and enable Jenkins:

```
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

- Access Jenkins via a browser:
  http://server_ip:8080

- Unlock Jenkins using the initial admin password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

- Install recommended plugins and create an admin user.

Step 4: Configure Jenkins for PHP Deployment

- Install necessary Jenkins plugins:
  - **Git Plugin**: To pull code from a Git repository.
  - **Publish Over SSH**: To deploy files to the server.
- Configure global tools in Jenkins:
  - Set up Git and PHP under **Manage Jenkins > Global Tool Configuration**.
- Add SSH credentials for the server:
  - Go to **Manage Jenkins > Manage Credentials**.
  - Add a new SSH username and private key for the server.

Step 5: Create a Jenkins Pipeline

- Create a new pipeline job in Jenkins:
  - Go to **New Item > Pipeline**.
  - Name the pipeline (e.g., `php-deployment`).
- Configure the pipeline:
  - Under **Pipeline**, select **Pipeline script from SCM**.
  - Choose **Git** as the SCM and provide the repository URL.
  - Specify the branch (e.g., `main` or `master`).
  - Set the script path to `Jenkinsfile` (this file will define the pipeline steps).

Step 6: Write the Jenkinsfile

- Create a `Jenkinsfile` in your Git repository to define the pipeline stages:

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url:
'<https://github.com/your-repo/php-app.git>'
            }
        }

        stage('Test') {
            steps {
                sh 'phpunit --configuration phpunit.xml'
            }
        }
```

```
        stage('Deploy') {
            steps {
                sshPublisher(
                    publishers: [
                        sshPublisherDesc(
                            configName: 'your-ssh-server',
                            transfers: [
                                sshTransfer(
                                    sourceFiles: '**/*',
                                    removePrefix: 'src',
                                    remoteDirectory: '/var/www/html',
                                    execCommand: 'sudo systemctl restart apache2'
                                )
                            ]
                        )
                    ]
                )
            }
        }
    }
}
```

Step 7: Run the Pipeline

- Manually trigger the pipeline or configure it to run automatically on code changes.
- Jenkins will:
    1. Pull the latest code from the Git repository.
    2. Run tests (e.g., PHPUnit).
    3. Deploy the application to the server using SSH.

Step 8: Verify Deployment

- Open a browser and navigate to your server's IP or domain:
  http://server_ip/

---

2. Benefits of CI/CD (Continuous Integration & Continuous Deployment)

CI/CD is a modern software development practice that automates the process of integrating code changes and deploying applications. Here are the key benefits:

1. Faster Development Cycles

- CI/CD automates testing and deployment, reducing manual effort and speeding up the release process.
- Developers can focus on writing code while CI/CD pipelines handle repetitive tasks.

2. Improved Code Quality

- Automated testing ensures that code changes are validated before being merged into the main branch.
- Bugs and issues are caught early, reducing the risk of deploying faulty code.

3. Reduced Deployment Risks

- Continuous Deployment ensures that small, incremental changes are deployed frequently, reducing the risk of large, complex deployments.
- Rollback mechanisms can be implemented to revert changes quickly if something goes wrong.

4. Enhanced Collaboration

- CI/CD encourages collaboration between development, testing, and operations teams.
- Everyone works on the same pipeline, ensuring consistency and transparency.

5. Scalability

- CI/CD pipelines can handle multiple environments (e.g., development, staging, production), making it easier to scale applications.

6. Cost Efficiency

- Automating repetitive tasks reduces the need for manual intervention, saving time and resources.
- Early detection of issues minimizes the cost of fixing bugs in production.

7. Better Customer Satisfaction

- Frequent updates and faster bug fixes lead to a better user experience.
- New features and improvements are delivered to users more quickly.

8. Auditability and Traceability

- CI/CD pipelines provide logs and records of every change, making it easier to track issues and understand the history of deployments.

Example CI/CD Workflow for a PHP Application

1. Continuous Integration (CI):
    - Developers push code to a Git repository (e.g., GitHub, GitLab).
    - A CI tool (e.g., Jenkins, GitHub Actions) automatically runs tests (unit tests, integration tests) on the new code.
    - If tests pass, the code is merged into the main branch.
2. Continuous Deployment (CD):
    - The CD pipeline automatically deploys the application to a staging environment for further testing.
    - Once approved, the application is deployed to the production environment.

Conclusion

- Deploying a PHP application on a Linux server involves setting up the server, configuring the web server and database, and uploading the application files.
- CI/CD provides significant benefits, including faster development cycles, improved code quality, and reduced deployment risks.
- By combining proper deployment practices with CI/CD, you can ensure a smooth and efficient software development lifecycle.